

Apply filters to SQL queries

Project description

My organization is actively enhancing its system security. As part of my role, I'm responsible for safeguarding the infrastructure, identifying and investigating potential vulnerabilities, and ensuring employee devices remain up to date. The steps below illustrate how I applied SQL filtering techniques to carry out key security-related tasks.

Retrieve after hours failed login attempts

A potential security incident was identified outside of standard business hours (post 18:00). All failed login attempts occurring during this after-hours period require thorough investigation. The SQL query below illustrates the method used to isolate these failed login events based on timestamp criteria.

```
MariaDB [organization]> SELECT *
->
-> FROM log_in_attempts
->
-> WHERE login_time > '18:00' AND success = FALSE;
```

event_id	username	login_date	login_time	country	ip_address	success
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
18	pwashing	2022-05-11	19:28:50	US	192.168.66.142	0
20	tshah	2022-05-12	18:56:36	MEXICO	192.168.109.50	0
28	aestrada	2022-05-09	19:28:12	MEXICO	192.168.27.57	0
34	drosas	2022-05-11	21:02:04	US	192.168.45.93	0
42	cgriffin	2022-05-09	23:04:05	US	192.168.4.157	0
52	cjackson	2022-05-10	22:07:07	CAN	192.168.58.57	0

The screenshot consists of two sections: the upper portion displays the SQL query, while the lower portion shows a subset of the resulting output. This query is designed to identify failed login attempts occurring after 18:00.

To construct the query, I began by selecting all records from the `log_in_attempts` table. I then applied a `WHERE` clause with an `AND` operator to narrow the results to entries meeting both of the following criteria:

- `login_time > '18:00'` — filters for login attempts initiated after 18:00
- `success = FALSE` — isolates attempts that were unsuccessful

This approach ensures that only after-hours failed login events are returned for further analysis.

Retrieve login attempts on specific dates

A potentially suspicious event was detected on 2022-05-09. As part of the investigation, all login activity from that date and the preceding day 2022-05-08 must be reviewed.

The SQL query below illustrates the approach used to filter login attempts occurring on these specific dates.

```
MariaDB [organization]> SELECT *  
->  
-> FROM log_in_attempts  
->  
-> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	1
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	1
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0
8	bisles	2022-05-08	01:30:17	US	192.168.119.173	0

To construct the query, I began by selecting all entries from the `log_in_attempts` table. I then applied a `WHERE` clause using the `OR` operator to filter the results based on two specific conditions:

- `login_date = '2022-05-09'` — captures login activity on May 9, 2022
- `login_date = '2022-05-08'` — captures login activity on May 8, 2022

This approach ensures that only login attempts from the relevant dates are included for further review.

Retrieve login attempts outside of Mexico

Following a review of the organization's login activity data, I identified a potential concern involving login attempts originating from outside Mexico. These entries warrant further investigation to assess any associated risks.

The SQL query below demonstrates the method used to isolate login attempts based on geographic location, specifically excluding those from Mexico.

```

MariaDB [organization]> SELECT *
->
-> FROM log_in_attempts
->
-> WHERE NOT country LIKE 'MEX%';

```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	1
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	1
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0

The screenshot is divided into two sections: the upper portion displays the SQL query, while the lower portion presents a sample of the resulting output. This query is designed to retrieve all login attempts originating from countries other than Mexico.

To construct the query, I began by selecting all records from the `log_in_attempts` table. I then applied a `WHERE` clause using the `NOT` and `LIKE` operator to exclude entries where the country field begins with "MEX%". This pattern was chosen to account for variations in how Mexico is represented in the dataset—specifically as "MEX" and "MEXICO". The wildcard allows for matching any sequence of characters following "MEX", ensuring comprehensive filtering. The percentage sign (%) represents any number of unspecified characters when used with `LIKE`.

Retrieve employees in Marketing

My team wants to upgrade computer systems for selected employees within the Marketing department. To facilitate this process, I first needed to identify which employee machines are eligible for the update.

The SQL query below demonstrates how I filtered the dataset to retrieve machines assigned to employees in the Marketing department located in the East building.

```
MariaDB [organization]> SELECT *  
->  
-> FROM employees  
->  
-> WHERE department = 'Marketing' AND office LIKE 'East%';  
+-----+-----+-----+-----+-----+  
| employee_id | device_id | username | department | office |  
+-----+-----+-----+-----+-----+  
| 1000 | a320b137c219 | elarson | Marketing | East-170 |  
| 1052 | a192b174c940 | jdarosa | Marketing | East-195 |  
| 1075 | x573y883z772 | fbautist | Marketing | East-267 |  
| 1088 | k865l965m233 | rgosh | Marketing | East-157 |  
| 1103 | NULL | randerss | Marketing | East-460 |  
| 1156 | a184b775c707 | dellery | Marketing | East-417 |  
| 1163 | h679i515j339 | cwilliam | Marketing | East-216 |  
+-----+-----+-----+-----+-----+  
7 rows in set (0.038 sec)  
  
MariaDB [organization]>
```

The first part of the screenshot is my query, and the second part is a portion of the output. This query returns all employees in the Marketing department in the East building. First, I started by selecting all data from the `employees` table. Then, I used a `WHERE` clause with `AND` to filter for employees who work in the Marketing department and in the East building. I used `LIKE` with `East%` as the pattern to match because the data in the `office` column represents the East building with the specific office number.

The first condition is the `department = 'Marketing'` portion, which filters for employees in the Marketing department. The second condition is the `office LIKE 'East%'` portion, which filters for employees in the East building.

Retrieve employees in Finance or Sales

Machines assigned to employees in the Finance and Sales departments also require a separate security update. To initiate this process, I must first retrieve data specifically for individuals within these two departments.

The SQL query below illustrates how I filtered for employee machines belonging exclusively to personnel in Finance or Sales.

```
MariaDB [organization]> SELECT *
->
-> FROM employees
->
-> WHERE department = 'Finance' OR department = 'Sales';
```

employee_id	device_id	username	department	office
1003	d394e816f943	sgilmore	Finance	South-153
1007	h174i497j413	wjaffrey	Finance	North-406
1008	i858j583k571	abernard	Finance	South-170

The screenshot consists of two sections: the top displays the query, and the bottom shows a portion of its output. This query retrieves all employees belonging to either the Finance or Sales departments. It begins by selecting all records from the `employees` table. A `WHERE` clause is then applied using the `OR` operator to include employees from both departments. The condition `department = 'Finance'` filters for Finance personnel, while `department = 'Sales'` captures those in Sales. The `OR` operator is used intentionally to ensure that employees from either department are returned, rather than requiring both conditions to be met simultaneously.

Retrieve all employees not in IT

We are preparing to implement an additional security update targeting employees outside the Information Technology department. To proceed, I must first identify and gather relevant data on these individuals.

The example below illustrates the SQL query I constructed to filter employee machines belonging to personnel not assigned to the Information Technology department.

```
MariaDB [organization]> SELECT *
->
-> FROM employees
->
-> WHERE NOT department = 'Information Technology';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1001	b239c825d303	bmoreno	Marketing	Central-276
1002	c116d593e558	tshah	Human Resources	North-434

The top section of the screenshot displays my SQL query, while the lower portion shows part of the resulting output. The query retrieves all employees who are not part of the Information Technology department. I began by selecting all records from the `employees` table, then applied a `WHERE` clause with the `NOT` operator to exclude entries associated with that department.

Summary

I constructed SQL queries with targeted filters to extract specific details regarding login attempts and associated employee machines. The queries utilized two distinct tables `log_in_attempts` and `employees` and incorporated logical operators such as `AND`, `OR`, `NOT` to isolate relevant data for each task. Additionally, I employed the `LIKE` operator in conjunction with the `%` wildcard to identify pattern-based matches within the dataset.